Algoritmos de ordenamientos

Programación II

1

3 Octubre

UNRN

Universidad Nacional
de **Río Negro**

r27

Programación I

Complejidad computacional

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

Prof. Daniel Teira/Diego Diaz

Sede Bariloche

Contenidos de la clase

- **Lo expuesto en estas diapositivas se sacó de *Estructuras de datos y Algoritmos de Aho, Hopcroft, Ullman.***

Tiempo de ejecución de un programa

- Una solución particular puede tener más de un algoritmo que lo resuelva: ¿cuál elijo?.
  - Algoritmo fácil de entender, codificar y depurar.
  - Algoritmo más críptico y eficiente, que ejecute con mayor rapidez.
- Se expresa la diyuntiva porque suele ser el caso, que un algoritmo más claro sea menos eficiente.

Una pregunta para responder el punto anterior puede ser: ¿qué tan frecuentemente lo voy a ejecutar?.

- Si la respuesta es *poco frecuente*: probablemente prefiera algo más claro.
- Si la respuesta es *muy frecuente*: probablemente prefiera algo más eficiente.

Tiempo de ejecución de un programa - Ejemplo: búsqueda.

```
bool buscarLineal(int vec[], int tam, int buscado)
{
     for (int i = 0; i < tam; i++)
     {
          if (vec[i] == buscado)
          {
               return true; // Encontrado
          }
     }
     return false; // No está
}
```

```
bool buscarBinaria(int vec[], int tam, int buscado)
{
     int inicio=0;
     int fin = tam-1;

     while(inicio<fin)
     {
          int medio = (fin-inicio)/2;
          if (vec[medio]==buscado)
               return true;
          if (vec[medio] < buscado)
               inicio=m+1;
          else
               fin = medio-1;
     }
     return false; // No está
}
```

Tiempo de ejecución de un programa - Ejemplo: búsqueda.

```
bool buscarLineal(int vec[], int tam, int buscado)
{
     for (int i = 0; i < tam; i++)
     {
          if (vec[i] == buscado)
          {
               return true; // Encontrado
          }
     }
     return false; // No está
}
```

```
bool buscarBinaria(int vec[], int tam, int buscado)
{
     int inicio=0;
     int fin = tam-1;

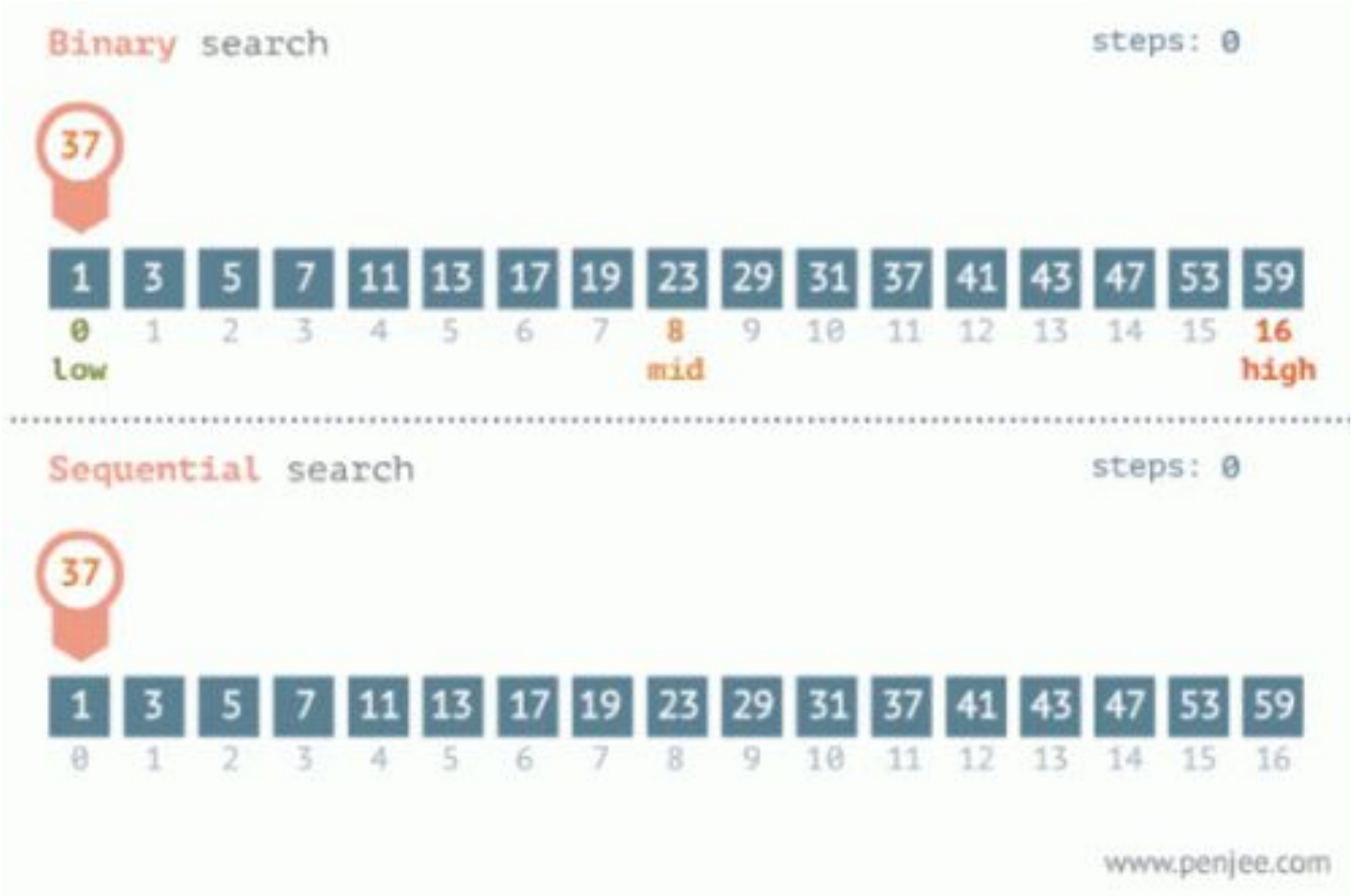
     while(inicio<fin)
     {
          int medio = (fin-inicio)/2;
          if (vec[medio]==buscado)
               return true;
          if (vec[medio] < buscado)
               inicio=m+1;
          else
               fin = medio-1;
     }
     return false; // No está
}
```

¿Cuál es más claro?

¿Cuál es más eficiente?

¿Cuál me conviene?

Tiempo de ejecución de un programa - Ejemplo: búsqueda.



Tiempo de ejecución de un programa.

- Factores determinantes del tiempo de ejecución:
  1. Los datos de entrada.
  2. La calidad del código generado.
  3. La naturaleza y velocidad del hardware.
  4. La complejidad temporal del algoritmo.

- Medición teórica ***T(n)***:

  El tiempo de ejecución se define como una función ***T(n)***, donde **n** representa el tamaño de la entrada (por ejemplo, número de elementos a ordenar).

  T(n) suele medirse en cantidad de instrucciones ejecutadas en una máquina idealizada para independizar la métrica del hardware específico.

Tiempo de ejecución de un programa - Ejemplo: búsqueda.

```
bool buscarLineal(int vec[], int tam, int buscado)
{
     for (int i = 0; i < tam; i++)
     {
          if (vec[i] == buscado)
          {
               return true; // Encontrado
          }
     }
     return false; // No está
}
```

```
bool buscarBinaria(int vec[], int tam, int buscado)
{
     int inicio=0;
     int fin = tam-1;

     while(inicio<fin)
     {
          int medio = (fin-inicio)/2;
          if (vec[medio]==buscado)
               return true;
          if (vec[medio] < buscado)
               inicio=m+1;
          else
               fin = medio-1;
     }
     return false; // No está
}
```

¿Qué variable va a modular la demora del algoritmo?

¿Tardan siempre igual para cualquier entrada?

Tiempo de ejecución de un programa - Ejemplo: búsqueda.

```
bool buscarLineal(int vec[], int tam, int buscado)
{
     for (int i = 0; i < tam; i++)
     {
          if (vec[i] == buscado)
          {
               return true; // Encontrado
          }
     }
     return false; // No está
}
```

- buscarLineal([1,2,3,4,5],5,1):
  - Sale en 1era iteración.
- buscarLineal([1,2,3,4,5],5,5)
  - Sale en la 5 iteración.
- buscarLineal([1,2,3,4,5],5,3)
  - Sale en la 3 iteración

Tiempo de ejecución de un programa - Ejemplo: búsqueda.

```
bool buscarBinaria(int vec[], int tam, int buscado)
{
     int inicio=0;
     int fin = tam-1;

     while(inicio<fin)
     {
          int medio = (fin-inicio)/2;
          if (vec[medio]==buscado)
               return true;
          if (vec[medio] < buscado)
               inicio=m+1;
          else
               fin = medio-1;
     }
     return false; // No está
}
```

- `buscarBinaria([1,2,3,4,5],5,1):`
  - `Sale en 3era iteración.`
- `buscarLineal([1,2,3,4,5],5,5)`
  - `Sale en la 3era iteración.`
- `buscarLineal([1,2,3,4,5],5,3)`
  - `Sale en la 1era iteración`

Tiempo de ejecución de un programa – Ejemplo: búsqueda.

- `buscarLineal([1,2,3,4,5],5,1)`:
  - Sale en 1era iteración.
- `buscarLineal([1,2,3,4,5],5,5)`
  - Sale en la 5 iteración.
- `buscarLineal([1,2,3,4,5],5,3)`
  - Sale en la 3 iteración

- `buscarBinaria([1,2,3,4,5],5,1)`:
  - Sale en 3era iteración.
- `buscarLineal([1,2,3,4,5],5,5)`
  - Sale en la 3era iteración.
- `buscarLineal([1,2,3,4,5],5,3)`
  - Sale en la 1era iteración

- Ejecuciones de Peor Caso - ***T(n)***:
  - `buscarLineal([1,2,3,4,5],5,5)`, `buscarBinaria([1,2,3,4,5],5,1)` y `buscarLineal([1,2,3,4,5],5,5)`
- Ejecuciones en caso promedio - $T_{prom}(n)$:
  - Cálculo estadístico. Si ejecuto muchas veces con entradas con distribución aleatoria uniforme.

Tiempo de ejecución de un programa.

- ***T(n)***: en general calcularemos el peor caso dado que el caso promedio no siempre es fácil de obtener y no siempre podemos asumir que las entradas son aleatorias o benignas.
- ***T(n)***: no podemos asociar T(n) con una unidad de tiempo o una demora porque la ejecución depende del compilador y las características del procesador donde se ejecute.
    - Por esto mismo, T(n) sirve para caracterizar y comparar algoritmos, independientemente de la plataforma donde se ejecute.

- ¿Cómo calculamos T(n) para un algoritmo?.

Tiempo de ejecución de un programa.

- ***T(n)***: en general calcularemos el peor caso dado que el caso promedio no siempre es fácil de obtener y no siempre podemos asumir que las entradas son aleatorias o benignas.
- ***T(n)***: no podemos asociar T(n) con una unidad de tiempo o una demora porque la ejecución depende del compilador y las características del procesador donde se ejecute.
    - Por esto mismo, T(n) sirve para caracterizar y comparar algoritmos, independientemente de la plataforma donde se ejecute.

- ¿Cómo calculamos T(n) para un algoritmo?.

Tiempo de ejecución de un programa.

- ***T(n)*** Es un tiempo exacto. Para hacer el cálculo más sencillo definimos ***O(n)*** (se lee “O-grande de n” u “O-mayúscula de n”).

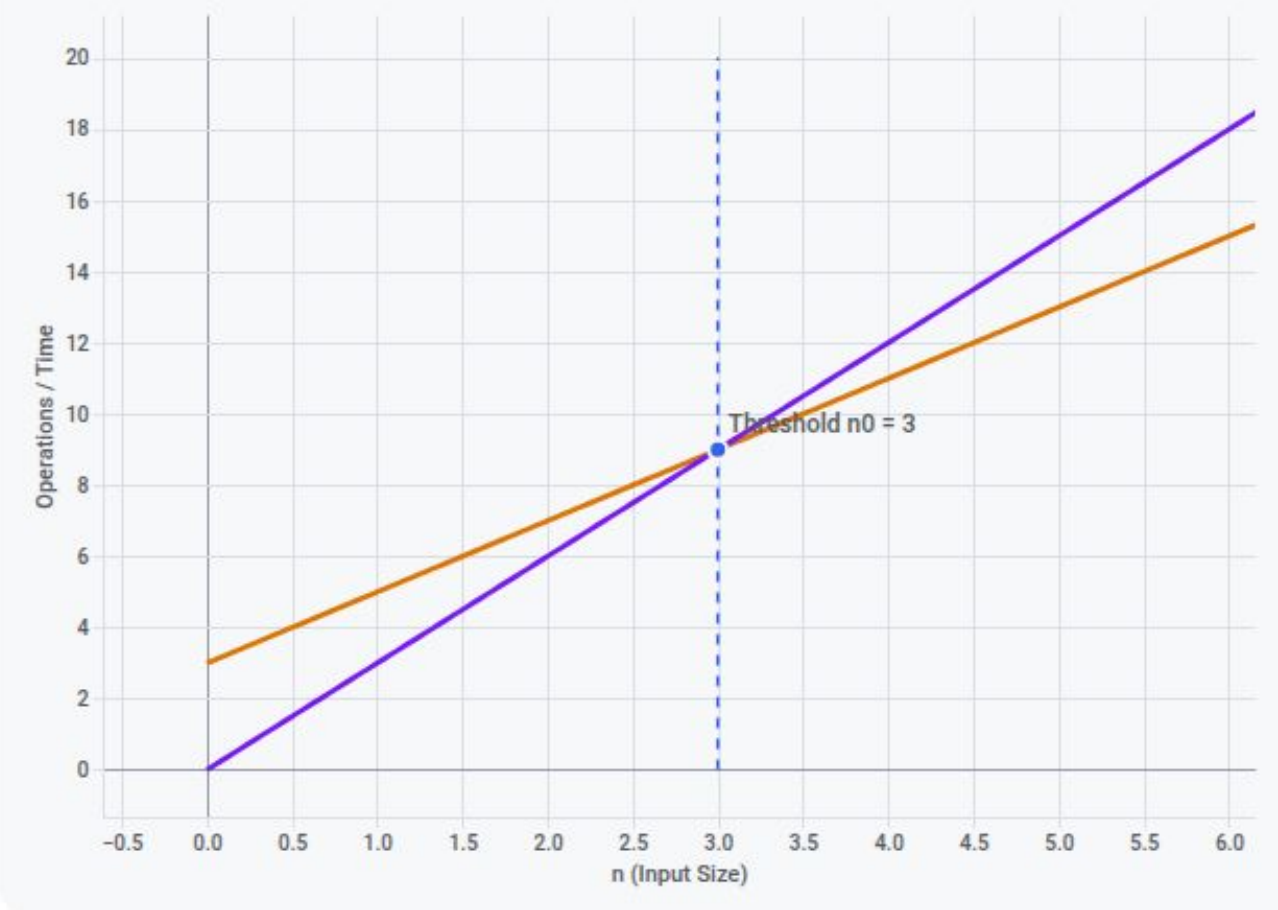
Entonces:

- *O(f(n))* indica que para un n_0 suficientemente grande, existe un *c* tal que: $T(n) < c * f(n)$ *para todo* $n > n_0$

    - Ejemplo:
        - (2n+3) (2 ciclos seguidos y 3 sentencias constantes)

Tiempo de ejecución de un programa.

- Ejemplo: 2n+3 vs 3n
  - *c = 3 y para n>3, 3n>*



Tiempo de ejecución de un programa.

- ¿Cómo calculamos T(n)?
  - Regla de la suma:
    - Para dos bloques de programa ejecutados en secuencia con tiempos *O(f(n)) y O(g(n))*, el tiempo de ejecución combinado es *O(max(f(n), g(n)))*.
    - En una secuencia fija de pasos, el tiempo de ejecución queda determinado por el paso de mayor costo.

```
programa()
{
        bloque1 -> n
        bloque2-> n²
        bloque3-> 5
}
O(max(n,n2,5))- >n2
```

Tiempo de ejecución de un programa.

- ¿Cómo calculamos T(n)?
  - Regla del producto:
    - Para construcciones anidadas con tiempos O(f(n)) y O(g(n)), el tiempo de ejecución resultante es O(f(n) * g(n)).

```
programa()
{
        while(cond) -> itera 5 veces
        {
                bloque1 -> n
        }
}  -> T(n) = 5*n. => O(T(n))) = O(5)*(O(n) = O(5*n)
```

    - Las constantes multiplicativas se omiten dentro de la notación: O(c * f(n)) equivale a O(f(n)).

        O(5*n) = O(n)

Tiempo de ejecución de un programa.

- ¿Cómo calculamos T(n)?
  - Regla del producto:
    - Para construcciones anidadas con tiempos O(f(n)) y O(g(n)), el tiempo de ejecución resultante es O(f(n) * g(n)).

```
programa()
{
        while(cond) -> itera n veces
        {
                bloque1 -> n
        }
}  -> T(n) = n*n. => O(T(n))) = O(n)*(O(n) = O(n*n)
```

    - Las constantes multiplicativas se omiten dentro de la notación: O(c * f(n)) equivale a O(f(n)).

      O(n*n) = O(n^2) => ¡¡¡ **n** no es una constante multiplicativa porque depende de la entrada!!!!

Tiempo de ejecución de un programa.

- Reglas Generales:

  a. Asignaciones y operaciones básicas:
  - Sentencias de asignación, lectura y escritura simples toman tiempo constante O(1).

  Condicionales (if-then-else):
  - El tiempo equivale al costo de evaluar la condición más el máximo tiempo de ejecución entre la rama de verdadero y la de falso.

  Bucles y ciclos:
  - El tiempo total de un ciclo es la suma, sobre todas las iteraciones, del tiempo necesario para ejecutar el cuerpo y evaluar la condición.
  - Si el tiempo del cuerpo es constante, se calcula multiplicando el número de iteraciones por la cota del cuerpo.

Tiempo de ejecución de un programa.

- Reglas Generales:

  Llamadas a Procedimientos (No Recursivos vs. Recursivos)

  - Procedimientos No Recursivos:
    - Se analizan de forma ascendente (bottom-up), resolviendo primero los procedimientos que no llaman a otros y sustituyendo sus tiempos en las llamadas.

Tiempo de ejecución de un programa.

- Reglas Generales:

  Llamadas a Procedimientos (No Recursivos vs. Recursivos)

  - Procedimientos No Recursivos:
    - Se analizan de forma ascendente (bottom-up), resolviendo primero los procedimientos que no llaman a otros y sustituyendo sus tiempos en las llamadas.

Tiempo de ejecución de un programa.

```
bool buscarLineal(int vec[], int tam, int buscado)
{
     for (int i = 0; i < tam; i++)
     {
          if (vec[i] == buscado)
          {
               return true; // Encontrado
          }
     }
     return false; // No está
}
```

Asignación, comparación, incremento y asignación	O(1).
Costo del if: costo de la rama mayor (O(1))+guarda (O(1)).	O(1).
Asignación	O(1).
El 'for' ejecuta `tam` veces.	* `tam`
Asignación	O(1)

`tam * (O(1)+...+O(1)) + O(1) = tam * O(1) = O(tam)`

donde `tam` e el tamaño del vector. Decimos que es un algoritmo de orden lineal.

Tiempo de ejecución de un programa.

```
bool buscarBinaria(int vec[], int tam, int buscado)
{
     int inicio=0;
     int fin = tam-1;

     while(inicio<fin)
     {
          int medio = (fin-inicio)/2;
          if (vec[medio]==buscado)
               return true;
          if (vec[medio] < buscado)
               inicio=m+1;
          else
               fin = medio-1;
     }
     return false; // No está
}
```

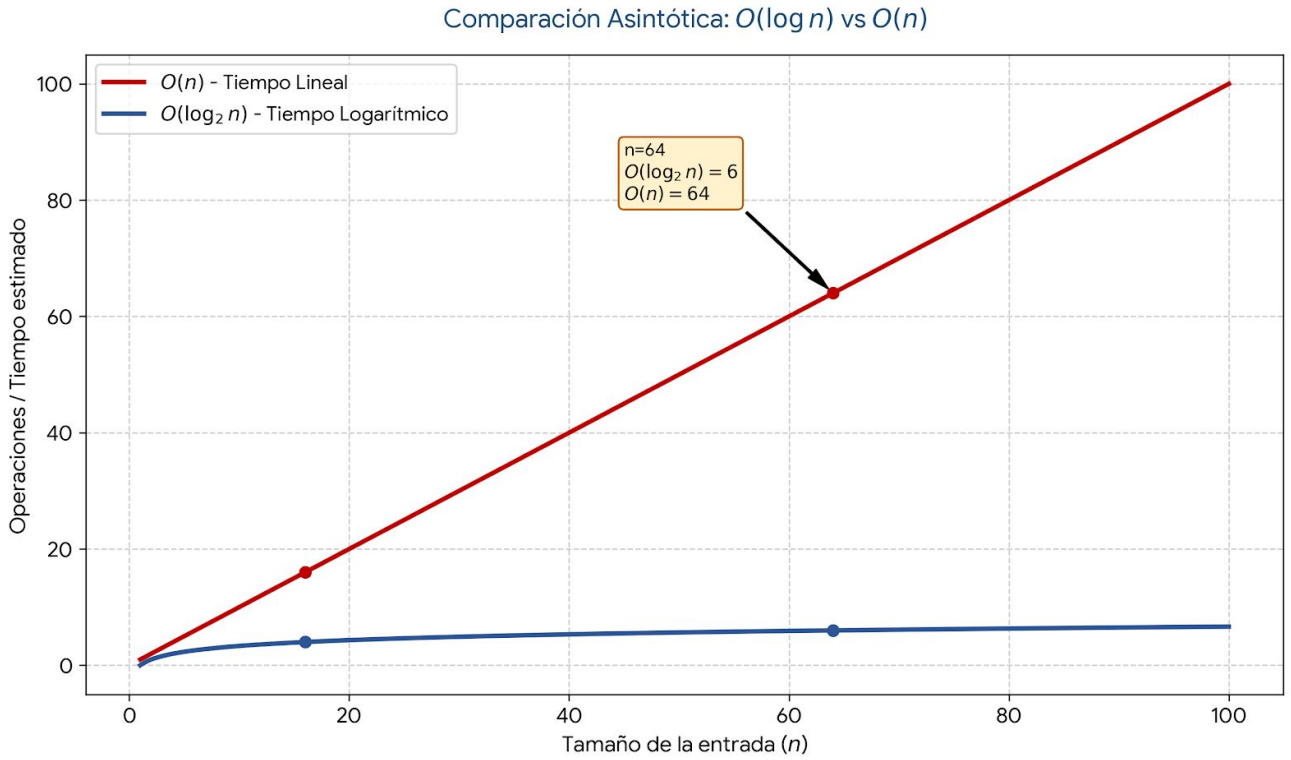
Asignación, comparación, incremento y asignación.	O(1)
Guarda O(1). Itera `log2(tam)` veces (Ver.)	O(1)
Asignación	O(1)
Condicional. Guarda O(1) y sentencia O(1).	O(1)
Condicional. Guarda O(1) y sentencia O(1).	O(1)
Asignación.	O(1)
Return	O(1)

`log(tam)*(O(1)+...+O(1))+O(1) = log(tam) * O(1) = O(log(tam))`

donde `tam` e el tamaño del vector. Decimos que es un algoritmo de orden logarítmico.

Tiempo de ejecución de un programa.

- ¿Cuál es más rápido, O(n) o O(log(n)).



n	log2(n)	2*n
1	0	2
5	2	10
10	3	20
15	4	30
20	4	40
40	5	80
100	7	200

Tiempo de ejecución de un programa.

```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarLineal(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

Guardas for	??
Costo del if: costo guarda	??
Costo del if: costo de la rama (no tiene else)	??
El ‘for’ ejecuta `tam1` veces.	??
Asignación	??

Tiempo de ejecución de un programa.

```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarLineal(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

Guardas for	O(1)
Costo del if: costo guarda	O(tam2)
Costo del if: costo de la rama (no tiene else)	O(1)
El 'for' ejecuta `tam1` veces.	O(1)
Asignación	O(1)

$$tam1*(O(1)+...+O(tam2))+O(1) = tam1 * O(tam2) = O(tam1*tam2))$$

Tiempo de ejecución de un programa.

```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarBinaria(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

Guardas for	??
Costo del if: costo guarda	??
Costo del if: costo de la rama (no tiene else)	??
El ‘for’ ejecuta `tam1` veces.	??
Asignación	??

Tiempo de ejecución de un programa.

```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarBinaria(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

Guardas for	O(1)
Costo del if: costo guarda	**O(log(tam2))**
Costo del if: costo de la rama (no tiene else)	O(1)
El 'for' ejecuta `tam1` veces.	O(1)
Asignación	O(1)

$$tam1*(O(1)+...+O(log(tam2)))+O(1) = tam1*O(log(tam2))$$
$$= O(tam1*log(tam2)))$$

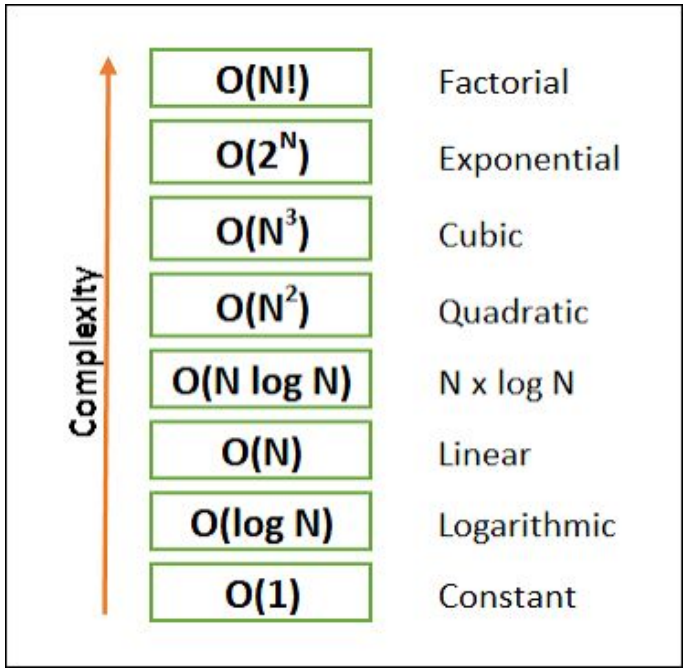
Tiempo de ejecución de un programa.

- ¿Cuál es más rápido, O(n*m) o O(log(n)*m).

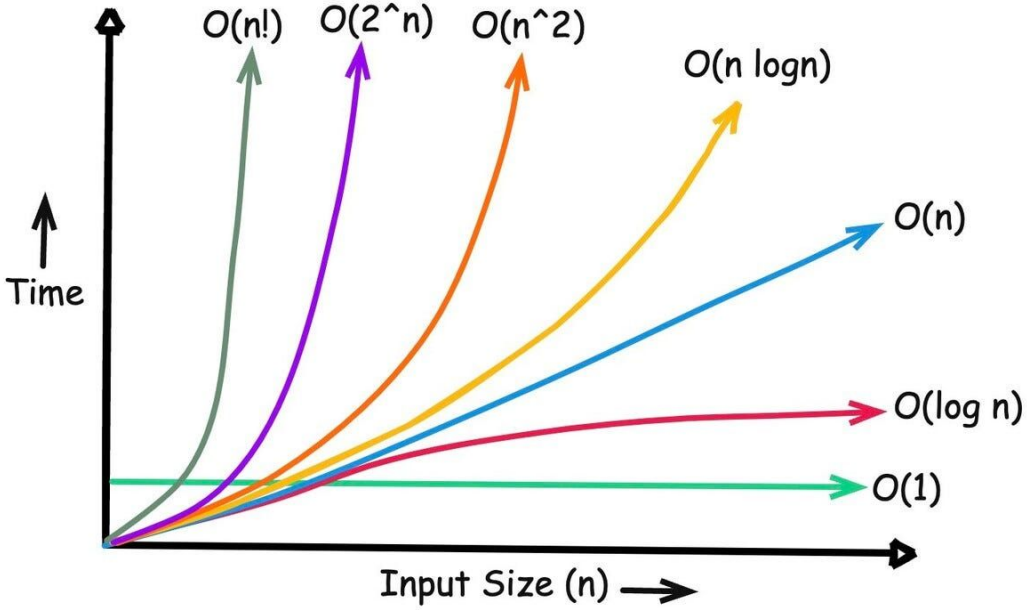
```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarBinaria(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

```
bool filtar(int vec1[], int tam1, int vec2[], int tam2)
{
     for (int i = 0; i < tam1; i++)
     {
          if (buscarLineal(vec2, tam2, vec1[i]) == true)
          {
               vec1[i]=-1;
          }
     }
     return false; // No está
}
```

Tiempo de ejecución de un programa. Comparemos



Function	Name
$\Theta(1)$	Constant
$\Theta(\log \log n)$	Log log
$\Theta(\log n)$	Logarithmic
$\Theta(n)$	Linear
$\Theta(n \log n)$	n log n
$\Theta(n^2)$	Quadratic
$\Theta(n^3)$	Cubic
$\Theta(c^n), c > 1$	Exponential
$\Theta(n!)$	Factorial



Tiempo de ejecución de un programa.

- Ejercicios

```
int obtener_suma_extremos(const int vec[], int n) {
    if (n <= 0) return 0;

    int primero = vec[0];
    int ultimo = vec[n - 1];

    return primero + ultimo;
}
```

Tiempo de ejecución de un programa.

- Ejercicios

```
long sumar_elementos(const int vec[], int n) {
    long suma = 0;

    for (int i = 0; i < n; i++) {
        suma += vec[i];
    }

    return suma;
```

Tiempo de ejecución de un programa.

- Ejercicios

```
bool tiene_duplicados(const int vec[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (vec[i] == vec[j]) {
                return true;
            }
        }
    }
    return false;
}
```